

Phase 2 Notes: Decreasing One-Call Recurrences

1. Main Goal of Phase 2

In Phase 1, you learned how to look at recursive code and write a recurrence relation.

The main recurrence from Phase 1 was:

$$\begin{aligned}T(n) &= T(n - 1) + 1 \\T(0) &= 1\end{aligned}$$

In Phase 2, the goal is to **solve** this recurrence.

That means we want to find the final time complexity.

So Phase 1 asks:

How do we write the recurrence?

Phase 2 asks:

How do we solve the recurrence?

The main method used in this phase is called:

successive substitution

It is also called:

repeated substitution

Both names mean the same thing.

2. Current Progress Analysis

Before starting Phase 2, you should already understand these Phase 1 ideas:

Skill	Meaning
Recognize recursion	You can see when a function calls itself.
Find the base case	You can find where recursion stops.
Find the recursive call	You can find the line where the function calls itself again.
Track input change	You can see that n becomes $n - 1$.
Count extra work	You can identify work outside recursion, such as printing once.
Write a simple recurrence	You can write $T(n) = T(n - 1) + 1$.
Understand basic Big O	You know this example gives $O(n)$ time.

Now you are ready to learn how to prove the answer using substitution.

3. What Is Covered in Phase 2?

Phase 2 covers these concepts:

1. Successive substitution method.
 2. Solving $T(n) = T(n - 1) + 1$.
 3. Finding the pattern after repeated substitution.
 4. Using the base case to stop the recurrence.
 5. Understanding why constant work becomes linear time.
 6. General decreasing one-call recurrence form.
 7. Understanding a , b , and $f(n)$.
 8. Solving examples with different extra work:
 9. $T(n) = T(n - 1) + 1$
 10. $T(n) = T(n - 1) + n$
 11. $T(n) = T(n - 1) + n^2$
 12. Common beginner mistakes.
 13. A simple solving checklist.
-

4. What Is a Decreasing One-Call Recurrence?

A recurrence is called **decreasing** when the input becomes smaller by subtraction.

For example:

n becomes n - 1

or:

n becomes n - 2

A recurrence is called **one-call** when the function makes only one recursive call.

Example:

```
printNumber(n - 1);
```

This is one recursive call.

So this type of recurrence is called:

decreasing one-call recurrence

because:

- the input decreases,
- and there is only one recursive call.

5. Phase 1 Example Code

The main example is:

```
#include <iostream>
using namespace std;

void printNumber(int n) {
    if (n > 0) {
        cout << n << " ";
        printNumber(n - 1);
    }
}

int main() {
    printNumber(5);
}
```

```
    return 0;  
}
```

This function prints numbers from `n` down to `1`.

For example:

```
printNumber(5);
```

prints:

```
5 4 3 2 1
```

6. Recurrence for the Code

Inside each call, two things happen.

Part 1: Current work

This line prints one number:

```
cout << n << " ";
```

Printing once is constant work.

So we write:

```
+1
```

Part 2: Recursive call

This line calls the function again:

```
printNumber(n - 1);
```

The input changes from:

n

to:

n - 1

So the recursive part is:

$T(n - 1)$

Full recurrence

$T(n) = T(n - 1) + 1$
 $T(0) = 1$

Meaning:

Part	Meaning
$T(n)$	Time for input size n
$T(n - 1)$	Time for the smaller recursive call
$+1$	Constant work done now
$T(0) = 1$	Base case takes constant time

7. What Does It Mean to Solve a Recurrence?

Solving a recurrence means finding the final running time.

For example, we start with:

$T(n) = T(n - 1) + 1$

But this is not the final answer yet.

It still contains another $T(\dots)$ term.

We want to simplify it until we get something like:

$$T(n) = n + 1$$

Then we can write the Big O answer:

$$O(n)$$

8. Successive Substitution Method

Successive substitution means replacing the smaller recurrence again and again.

Start with:

$$T(n) = T(n - 1) + 1$$

Now replace $T(n - 1)$ using the same recurrence rule.

Since:

$$T(n - 1) = T(n - 2) + 1$$

we substitute it:

$$T(n) = [T(n - 2) + 1] + 1$$

So:

$$T(n) = T(n - 2) + 2$$

Now replace $T(n - 2)$.

Since:

$$T(n - 2) = T(n - 3) + 1$$

we get:

$$T(n) = [T(n - 3) + 1] + 2$$

So:

$$T(n) = T(n - 3) + 3$$

Now we can see a pattern.

9. Substitution Table

Step	Expression
Start	$T(n) = T(n - 1) + 1$
After 1 substitution	$T(n) = T(n - 2) + 2$
After 2 substitutions	$T(n) = T(n - 3) + 3$
After k substitutions	$T(n) = T(n - k) + k$

The pattern is:

$$T(n) = T(n - k) + k$$

This means after k steps:

- the input becomes $n - k$,
- and the total added work becomes k .

10. Why Do We Use k ?

We use k to represent the number of substitution steps.

For example:

Number of steps	Result
1 step	$T(n) = T(n - 1) + 1$
2 steps	$T(n) = T(n - 2) + 2$
3 steps	$T(n) = T(n - 3) + 3$

Number of steps	Result
k steps	$T(n) = T(n - k) + k$

So k helps us write the general pattern.

But this is not the final answer yet.

We still need to reach the base case.

11. Reaching the Base Case

The base case is:

$$T(0) = 1$$

So we want this part:

$$T(n - k)$$

to become:

$$T(0)$$

That means:

$$n - k = 0$$

Now solve for k :

$$\begin{aligned} n - k &= 0 \\ k &= n \end{aligned}$$

So the recurrence reaches the base case after n steps.

12. Substituting $k = n$

We already found the pattern:

$$T(n) = T(n - k) + k$$

Now put:

$$k = n$$

So:

$$T(n) = T(n - n) + n$$

Simplify:

$$T(n) = T(0) + n$$

Since:

$$T(0) = 1$$

we get:

$$T(n) = 1 + n$$

So:

$$T(n) = n + 1$$

In Big O notation:

$$T(n) = O(n)$$

13. Full Solution in One Place

Start:

$$T(n) = T(n - 1) + 1$$

Substitute:

$$T(n) = T(n - 2) + 2$$

Substitute again:

$$T(n) = T(n - 3) + 3$$

Pattern:

$$T(n) = T(n - k) + k$$

Base case:

$$n - k = 0$$

Solve:

$$k = n$$

Put $k = n$ into the pattern:

$$T(n) = T(n - n) + n$$

So:

$$T(n) = T(0) + n$$

Since:

$$T(0) = 1$$

Final:

$$T(n) = 1 + n$$

Big O:

$$O(n)$$

14. Beginner-Friendly Meaning of the Solution

The recurrence:

$$T(n) = T(n - 1) + 1$$

means:

Do 1 unit of work now, then solve the same problem for $n - 1$.

The calls go like this:

```
n
n - 1
n - 2
n - 3
...
1
0
```

At each level, the function does 1 unit of work.

So the total work is:

$$1 + 1 + 1 + \dots + 1$$

This happens about n times.

Therefore:

$$T(n) = O(n)$$

15. Why Constant Work Becomes Linear Time

A beginner might think:

The function only does +1 work, so maybe it is $O(1)$.

But that is not correct.

The function does +1 work in every recursive call.

For example, for $n = 5$:

Call	Work
<code>printNumber(5)</code>	1
<code>printNumber(4)</code>	1
<code>printNumber(3)</code>	1
<code>printNumber(2)</code>	1
<code>printNumber(1)</code>	1

Total work:

$$1 + 1 + 1 + 1 + 1 = 5$$

For general n :

$$1 + 1 + 1 + \dots + 1 = n$$

So the time complexity is:

$$O(n)$$

16. General Decreasing Recurrence Form

The general form for decreasing recurrences is:

$$T(n) = aT(n - b) + f(n)$$

Meaning:

Symbol	Meaning
a	Number of recursive calls
b	Amount subtracted from n
$f(n)$	Extra work done outside recursion

For the main example:

$$T(n) = T(n - 1) + 1$$

we have:

Symbol	Value	Meaning
a	1	There is one recursive call.
b	1	The input decreases by 1.
$f(n)$	1	Each call does constant work.

So the recurrence can also be written as:

$$T(n) = 1T(n - 1) + 1$$

17. What Is $f(n)$?

$f(n)$ means the work done outside the recursive call.

Examples:

Example 1: Printing once

```
cout << n << " ";
```

This is constant work:

$$f(n) = 1$$

So the recurrence is:

$$T(n) = T(n - 1) + 1$$

Example 2: One loop from 1 to n

```
for (int i = 1; i <= n; i++) {  
    cout << i << " ";
```

```
}
```

This loop runs n times.

So:

$$f(n) = n$$

The recurrence is:

$$T(n) = T(n - 1) + n$$

Example 3: Nested loops

```
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= n; j++) {  
        cout << "*";  
    }  
}
```

This does about $n \times n$ work.

So:

$$f(n) = n^2$$

The recurrence is:

$$T(n) = T(n - 1) + n^2$$

18. Practice Pattern 1: Constant Work

Recurrence:

$$T(n) = T(n - 1) + 1$$

This means:

- one recursive call,
- input decreases by 1,
- each call does constant work.

Total work:

$$1 + 1 + 1 + \dots + 1$$

There are terms.

So:

$$T(n) = O(n)$$

19. Practice Pattern 2: Linear Work

Recurrence:

$$T(n) = T(n - 1) + n$$

This means:

- one recursive call,
- input decreases by 1,
- current call does n work.

The total work becomes:

$$n + (n - 1) + (n - 2) + \dots + 1$$

This is the sum of the first n numbers.

The formula is:

$$1 + 2 + 3 + \dots + n = n(n + 1) / 2$$

So:

$$T(n) = n(n + 1) / 2$$

In Big O:

$$T(n) = O(n^2)$$

Simple explanation

A loop of size n happens first.

Then a loop of size $n - 1$ happens.

Then a loop of size $n - 2$ happens.

So the work piles up like this:

$$n + (n - 1) + (n - 2) + \dots + 1$$

That grows like n^2 .

20. Practice Pattern 3: Quadratic Work

Recurrence:

$$T(n) = T(n - 1) + n^2$$

This means:

- one recursive call,
- input decreases by 1,
- current call does n^2 work.

The total work becomes:

$$n^2 + (n - 1)^2 + (n - 2)^2 + \dots + 1^2$$

This is the sum of squares.

The exact formula is:

$$1^2 + 2^2 + 3^2 + \dots + n^2 = n(n + 1)(2n + 1) / 6$$

The largest growth term is n^3 .

So:

$$T(n) = O(n^3)$$

Simple explanation

If each call does square work, and there are many calls, the total becomes cubic.

So:

$$T(n) = T(n - 1) + n^2$$

has:

$$O(n^3)$$

21. Important Pattern Summary

For decreasing one-call recurrences:

$$T(n) = T(n - 1) + f(n)$$

The final time is found by adding the work from all levels:

$$f(n) + f(n - 1) + f(n - 2) + \dots + f(1)$$

Recurrence	Total work	Final time
$T(n) = T(n - 1) + 1$	$1 + 1 + \dots + 1$	$O(n)$
$T(n) = T(n - 1) + n$	$n + (n - 1) + \dots + 1$	$O(n^2)$
$T(n) = T(n - 1) + n^2$	$n^2 + (n - 1)^2 + \dots + 1^2$	$O(n^3)$

Simple rule:

constant work $\rightarrow O(n)$
linear work $\rightarrow O(n^2)$
quadratic work $\rightarrow O(n^3)$

22. Complete Coding Program for Phase 2

This program shows three decreasing one-call functions.

```
#include <iostream>
using namespace std;

void constantWork(int n) {
    if (n > 0) {
        cout << n << " ";    // O(1) work
        constantWork(n - 1);  // one recursive call
    }
}

void linearWork(int n) {
    if (n > 0) {
```

```

        for (int i = 1; i <= n; i++) {
            cout << i << " ";    // O(n) work in this call
        }
        cout << "| ";
        linearWork(n - 1);        // one recursive call
    }
}

void quadraticWork(int n) {
    if (n > 0) {
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                cout << "*";    // O(n²) work in this call
            }
        }
        cout << "| ";
        quadraticWork(n - 1);    // one recursive call
    }
}

int main() {
    int n = 3;

    cout << "Constant work: ";
    constantWork(n);

    cout << "\nLinear work: ";
    linearWork(n);

    cout << "\nQuadratic work: ";
    quadraticWork(n);

    cout << endl;
    return 0;
}

```

23. Dry Run for `n = 3`

Function 1: `constantWork(3)`

Calls:

```

constantWork(3)
constantWork(2)

```

```
constantWork(1)
constantWork(0)
```

Work done:

$$1 + 1 + 1 = 3$$

So for general n :

$$O(n)$$

Function 2: `linearWork(3)`

Work by call:

Call	Work
<code>linearWork(3)</code>	3 operations
<code>linearWork(2)</code>	2 operations
<code>linearWork(1)</code>	1 operation

Total:

$$3 + 2 + 1 = 6$$

For general n :

$$n + (n - 1) + \dots + 1 = O(n^2)$$

Function 3: `quadraticWork(3)`

Work by call:

Call	Work
<code>quadraticWork(3)</code>	$3^2 = 9$ operations
<code>quadraticWork(2)</code>	$2^2 = 4$ operations

Call	Work
<code>quadraticWork(1)</code>	<code>1² = 1</code> operation

Total:

$$9 + 4 + 1 = 14$$

For general `n`:

$$n^2 + (n - 1)^2 + \dots + 1^2 = O(n^3)$$

24. Time Complexity vs Stack Space

In Phase 2, the input usually decreases like this:

$$n \rightarrow n - 1 \rightarrow n - 2 \rightarrow \dots \rightarrow 1 \rightarrow 0$$

That means the recursion depth is about `n`.

So the stack space is:

$$O(n)$$

This is true for these one-call decreasing examples:

Recurrence	Time Complexity	Stack Space
<code>T(n) = T(n - 1) + 1</code>	<code>O(n)</code>	<code>O(n)</code>
<code>T(n) = T(n - 1) + n</code>	<code>O(n²)</code>	<code>O(n)</code>
<code>T(n) = T(n - 1) + n²</code>	<code>O(n³)</code>	<code>O(n)</code>

The time changes because the work per call changes.

But the stack space stays `O(n)` because there is still only one chain of recursive calls.

25. Why Stack Space Is Still $O(n)$

For one-call recursion, the calls form a chain:

```
function(n)
  function(n - 1)
    function(n - 2)
      function(n - 3)
        ...
          function(0)
```

At the deepest point, about n calls are waiting.

So stack memory is:

$O(n)$

Even if the time is $O(n^2)$ or $O(n^3)$, the stack space is still $O(n)$ for these examples.

26. Common Beginner Mistakes

Mistake 1: Stopping substitution too early

Some students stop at:

$$T(n) = T(n - k) + k$$

But this is not the final answer.

You must continue until the base case.

You need to solve:

$$n - k = 0$$

Then:

$$k = n$$

Mistake 2: Forgetting the base case

The base case is very important.

For the main recurrence:

$$T(0) = 1$$

Without the base case, we do not know when to stop substituting.

Mistake 3: Thinking $+n$ always means $O(n)$

This is wrong:

$$T(n) = T(n - 1) + n$$

The $+n$ does not happen only once.

It happens at many levels:

$$n + (n - 1) + (n - 2) + \dots + 1$$

So the answer is:

$$O(n^2)$$

Mistake 4: Ignoring the work done in the current call

Some students write only:

$$T(n) = T(n - 1)$$

But if the function prints once, there is extra work.

Correct:

$$T(n) = T(n - 1) + 1$$

Mistake 5: Confusing output with time complexity

The output for `printNumber(5)` is:

5 4 3 2 1

But time complexity is not the printed output.

Time complexity is about how many operations happen.

For this function, there are about `n` working calls.

So:

$O(n)$

Mistake 6: Ignoring recursion stack space

Recursive functions use memory.

Even when the time is $O(n)$, the stack space may also be:

$O(n)$

because many calls wait on the call stack.

27. Step-by-Step Checklist for Solving Phase 2 Recurrences

When you see a decreasing one-call recurrence, follow these steps.

Step 1: Write the recurrence

Example:

$$T(n) = T(n - 1) + 1$$

Step 2: Substitute repeatedly

$$\begin{aligned} T(n) &= T(n - 2) + 2 \\ T(n) &= T(n - 3) + 3 \end{aligned}$$

Step 3: Find the pattern

$$T(n) = T(n - k) + k$$

Step 4: Use the base case

If the base case is $T(0)$, set:

$$n - k = 0$$

Step 5: Solve for k

$$k = n$$

Step 6: Substitute k back

$$T(n) = T(0) + n$$

Step 7: Use the base case value

$$T(0) = 1$$

So:

$$T(n) = 1 + n$$

Step 8: Write Big O

$$T(n) = O(n)$$

28. Mini Practice Questions

Practice 1

$$T(n) = T(n - 1) + 1$$

Answer:

$$O(n)$$

Reason:

$$1 + 1 + 1 + \dots + 1 = n$$

Practice 2

$$T(n) = T(n - 1) + n$$

Answer:

$$O(n^2)$$

Reason:

$$n + (n - 1) + (n - 2) + \dots + 1 = O(n^2)$$

Practice 3

$$T(n) = T(n - 1) + n^2$$

Answer:

$$O(n^3)$$

Reason:

$$n^2 + (n - 1)^2 + (n - 2)^2 + \dots + 1^2 = O(n^3)$$

Practice 4

$$T(n) = T(n - 1) + 5$$

Answer:

$$O(n)$$

Reason:

Each call does constant work.

The constant is **5**, but Big O ignores constant factors.

So:

$$5 + 5 + 5 + \dots + 5 = 5n = O(n)$$

Practice 5

$$T(n) = T(n - 1) + 3n$$

Answer:

$$O(n^2)$$

Reason:

3n is linear work.

Big O ignores the constant **3**.

So it behaves like:

$$T(n) = T(n - 1) + n$$

Final answer:

$$O(n^2)$$

29. Simple Way to Remember Phase 2

For this recurrence:

$$T(n) = T(n - 1) + 1$$

think:

The function walks down from n to 0.
Each step costs 1.
There are n steps.
So the answer is $O(n)$.

For this recurrence:

$$T(n) = T(n - 1) + n$$

think:

The function walks down from n to 0.
But the work at each step is large:
 $n, n - 1, n - 2, \dots, 1$.
So the answer is $O(n^2)$.

For this recurrence:

$$T(n) = T(n - 1) + n^2$$

think:

The function walks down from n to 0.
Each level does square work.
So the total becomes $O(n^3)$.

30. Final Phase 2 Summary

In Phase 2, you learned how to solve decreasing one-call recurrences.

The main recurrence is:

$$\begin{aligned}T(n) &= T(n - 1) + 1 \\T(0) &= 1\end{aligned}$$

Using successive substitution:

$$\begin{aligned}T(n) &= T(n - 1) + 1 \\T(n) &= T(n - 2) + 2 \\T(n) &= T(n - 3) + 3\end{aligned}$$

Pattern:

$$T(n) = T(n - k) + k$$

Use the base case:

$$\begin{aligned}n - k &= 0 \\k &= n\end{aligned}$$

Final:

$$\begin{aligned}T(n) &= T(0) + n \\T(n) &= 1 + n \\T(n) &= O(n)\end{aligned}$$

You also learned these important patterns:

Recurrence	Final Time
$T(n) = T(n - 1) + 1$	$O(n)$
$T(n) = T(n - 1) + n$	$O(n^2)$
$T(n) = T(n - 1) + n^2$	$O(n^3)$

The key idea is:

Writing the recurrence is only the first step.
Solving the recurrence tells us the final time complexity.

31. What You Should Be Able to Do After Phase 2

After Phase 2, you should be able to:

1. Solve $T(n) = T(n - 1) + 1$ using substitution.
2. Understand why the answer is $O(n)$.
3. Find the pattern $T(n) = T(n - k) + k$.
4. Use the base case equation $n - k = 0$.
5. Solve for k .
6. Substitute back into the recurrence.
7. Understand how extra work changes the final complexity.
8. Recognize these patterns:
 9. constant work gives $O(n)$,
 10. linear work gives $O(n^2)$,
 11. quadratic work gives $O(n^3)$.
12. Remember that stack space is still $O(n)$ for one-call decreasing recursion.
13. Avoid common mistakes like stopping substitution too early.

32. Key Formula From Phase 2

Main formula:

$$T(n) = T(n - 1) + 1$$

After substitution:

$$T(n) = T(n - k) + k$$

Base case:

$$\begin{aligned}n - k &= 0 \\ k &= n\end{aligned}$$

Final result:

$$\begin{aligned}T(n) &= T(0) + n \\ T(n) &= 1 + n \\ T(n) &= O(n)\end{aligned}$$

This is the foundation for Phase 3, where the function makes more than one recursive call.